

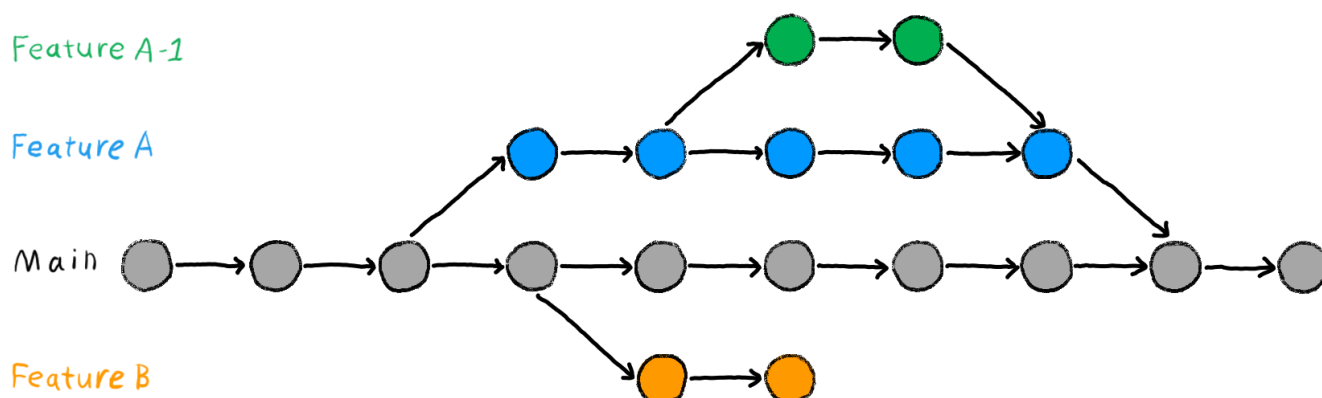
Trabalhando com branches

Na aula anterior aprendemos sobre a história do Git e os problemas que ele propôs resolver. Além disso também aprendemos como criar um repositório e enviar atualizações pra ele. Na aula de hoje veremos mais sobre as **BRANCHS**, uma ferramenta essencial no Git.

O que são as branches?

O próprio nome já indica seu significado. Branch, do inglês, significa ramo. Criar uma branch em um repositório é, portanto, criar um novo ramo do código, permitindo que diferentes versões do projeto evoluam de forma paralela.

As branches são usadas para criar um caminho paralelo ao código principal, seja porque várias pessoas estão trabalhando no mesmo projeto ou porque diferentes funcionalidades estão sendo desenvolvidas ao mesmo tempo. Dessa forma, cada desenvolvedor ou cada nova ideia pode evoluir sem interferir diretamente no funcionamento do sistema principal.



Por padrão, todo repositório Git possui uma branch principal, normalmente chamada de **main** ou, em projetos mais antigos, **master**. Essa branch representa a versão estável do projeto, aquela que está pronta para uso, testes ou deploy. Em geral, evita-se trabalhar diretamente nela, justamente para não comprometer o funcionamento do código principal.

A ideia ao criar uma branch é fazer uma cópia do repositório a partir de um determinado commit da branch main e, a partir disso, iniciar uma nova sequência de modificações. Assim, é possível desenvolver novas funcionalidades, corrigir bugs ou refatorar partes do código sem alterar o fluxo principal do projeto. Quando o trabalho estiver finalizado e testado, essas alterações podem ser integradas novamente à branch main.

Observação: Novas branches podem ser criadas a partir de outras branches e não necessariamente da main

Em um projeto organizado, cada nova implementação, correção de bug ou refatoração costuma ter a sua própria branch. No laboratório, em geral seguimos o seguinte padrão de nomenclatura:

```
nome.sobrenome/tipo/nova-implementacao
```

O campo tipo indica o objetivo da branch. Pode ser **dev** para desenvolvimento genérico, **feat** para novas funcionalidades, **fix** para correção de bugs ou **docs** para documentação. Existem várias outras possibilidades e, no fim, o nome da branch pode ser adaptado conforme a necessidade do projeto. Essas são apenas convenções amplamente utilizadas para facilitar a organização e o trabalho em equipe.

Ex: `joao.garcia/dev/global-planner`

Operações com branch

Antes de começar a trabalhar na prática com branches, é importante entender quais são as principais operações que podemos realizar com elas dentro do Git.

Criar branch

A operação mais básica é a **criação de uma branch**. Criar uma nova branch significa iniciar uma nova linha de desenvolvimento a partir de um ponto específico do projeto, geralmente a partir da branch main. A partir desse momento, todas as alterações feitas ficam isoladas nessa branch, sem afetar o código principal.

```
git branch joao.garcia/cpp/lista-1
```

Trocar branch

Outra operação fundamental é **trocar de branch**. O Git permite alternar entre diferentes branches do repositório, mudando o contexto de trabalho. Isso possibilita, por exemplo, pausar o desenvolvimento de uma funcionalidade e corrigir um bug em outra branch, retornando depois ao trabalho original sem misturar alterações.

```
git checkout joao.garcia/python/lista-1
```

Ou se deseja criar uma nova branch a partir da branch em que você está trabalhando e já mudar para ela:

```
git checkout -b joao.garcia/python/lista-2
```

Outra opção mais recente (desde 2019) para alternar entre branches é o **git switch**, que acaba sendo menos utilizado principalmente por hábito dos programadores e não por ser ruim. No caso ele é até mais claro a respeito de trocar de branches.

```
git switch joao.garcia/python/lista-1
```

Listar branch

Também é muito comum é **listar as branches** existentes no repositório, tanto locais quanto remotas. Isso ajuda a ter uma visão geral do projeto, identificar quais branches estão ativas e saber em qual delas você está

trabalhando no momento.

```
git branch
```

Deletar branch

Também podemos **remover uma branch** quando ela não é mais necessária. Normalmente isso acontece depois que a branch já foi unificada com a main ou quando uma implementação foi descartada. Deletar branches antigas ajuda a manter o repositório organizado.

```
git branch -d joao.garcia/dev/lista-2
```

Unificar branch

Outra operação importante é **unificar branches**, operação conhecida como **merge**. O merge é utilizado quando queremos integrar as alterações de uma branch em outra. Esse processo preserva o histórico de commits e consolida o trabalho realizado em paralelo.

```
git checkout main          # muda para a branch main
git merge nome-da-branch   # Unifica a branch escolhida na main
```

IMPORTANTE: O processo de merge envolve lidar com conflitos e inconsistências entre duas versões de código. Vamos abordar isso mais a frente na aula.

Enviar ou baixar uma branch do repo remoto

Por fim, em projetos colaborativos, é essencial **enviar e baixar branches do repositório remoto**. Isso permite compartilhar seu trabalho com outras pessoas da equipe e manter sua branch sincronizada com alterações feitas por outros desenvolvedores.

Apenas buscar as atualizações sem integrar automaticamente, usando o **git fetch**. Esse comando baixa as informações do repositório remoto, mas não altera seu código imediatamente, o que é útil para inspecionar mudanças antes de integrá-las.

```
git fetch
```

Enviar uma branch específica que ainda não existe no repositório remoto (ex: github)

```
git push -u origin nome-da-branch
```

Puxar uma branch específica que ainda não existe no repositório local (no seu PC).

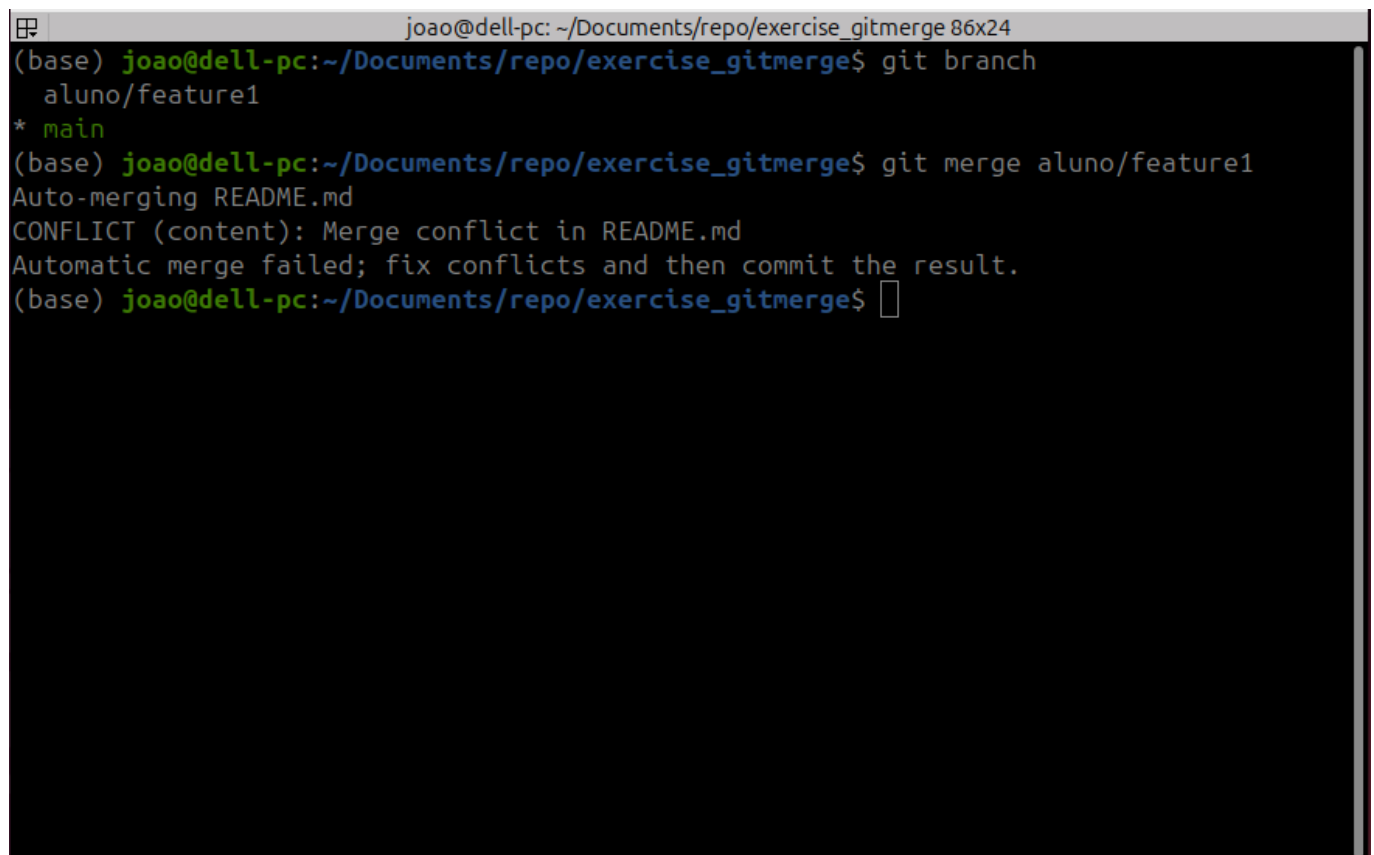
```
git fetch origin
git checkout -b nome-da-branch origin/nome-da-branch
```

De resto caso as branches já existe no repositório remoto e local o básico `git push` e `git pull` funcionam.

Se deseja listar todas as branches inclusive as informações que vem com o fetch, use `git branch --all`

Git merge: Lidando com conflitos

Em uma boa parte dos casos o `git merge` consegue resolver automaticamente a mesclagem das duas branches. Contudo quando trabalhando com multiplas branches pode acontecer de o mesmo arquivo ser alterado em ambas as branches, gerando um conflito entre elas. Veja o exemplo abaixo:

A terminal window with a dark background and light-colored text. The title bar shows 'joao@dell-pc: ~/Documents/repo/exercise_gitmerge 86x24'. The terminal output shows the following commands and responses:

```
(base) joao@dell-pc:~/Documents/repo/exercise_gitmerge$ git branch
aluno/feature1
* main
(base) joao@dell-pc:~/Documents/repo/exercise_gitmerge$ git merge aluno/feature1
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
(base) joao@dell-pc:~/Documents/repo/exercise_gitmerge$
```

Nesse caso, o Git tentou realizar o merge automatico do arquivo `README.md`, mas encontrou um conflito. Então ele pede para resolvermos o conflito e depois realizar o commit de merge. Se abirmos o arquivo `README.md`

```
joao@dell-pc: ~/Documents/repo/exercise_gitmerge 82x24
GNU nano 7.2 README.md
# Exercio de Merge

Esse repositório é utilizado como exemplo de merge para aula de Git "Criando Bran>
Nesse repo temos duas branchs, a `main` e a `aluno/feature1`. Na `main` encontram>
Para treinar seu objetivo é realizar o merge da branch `aluno/feature1` com a `ma>

<<<<<<< HEAD
## Alteração do mesmo arquivo

Essa é uma alteração do readme na `main`, o readme também foi alterado na branch >
=====
Essa é uma alteração no README feito pela branch `aluno/feature1`
>>>>>>> aluno/feature1

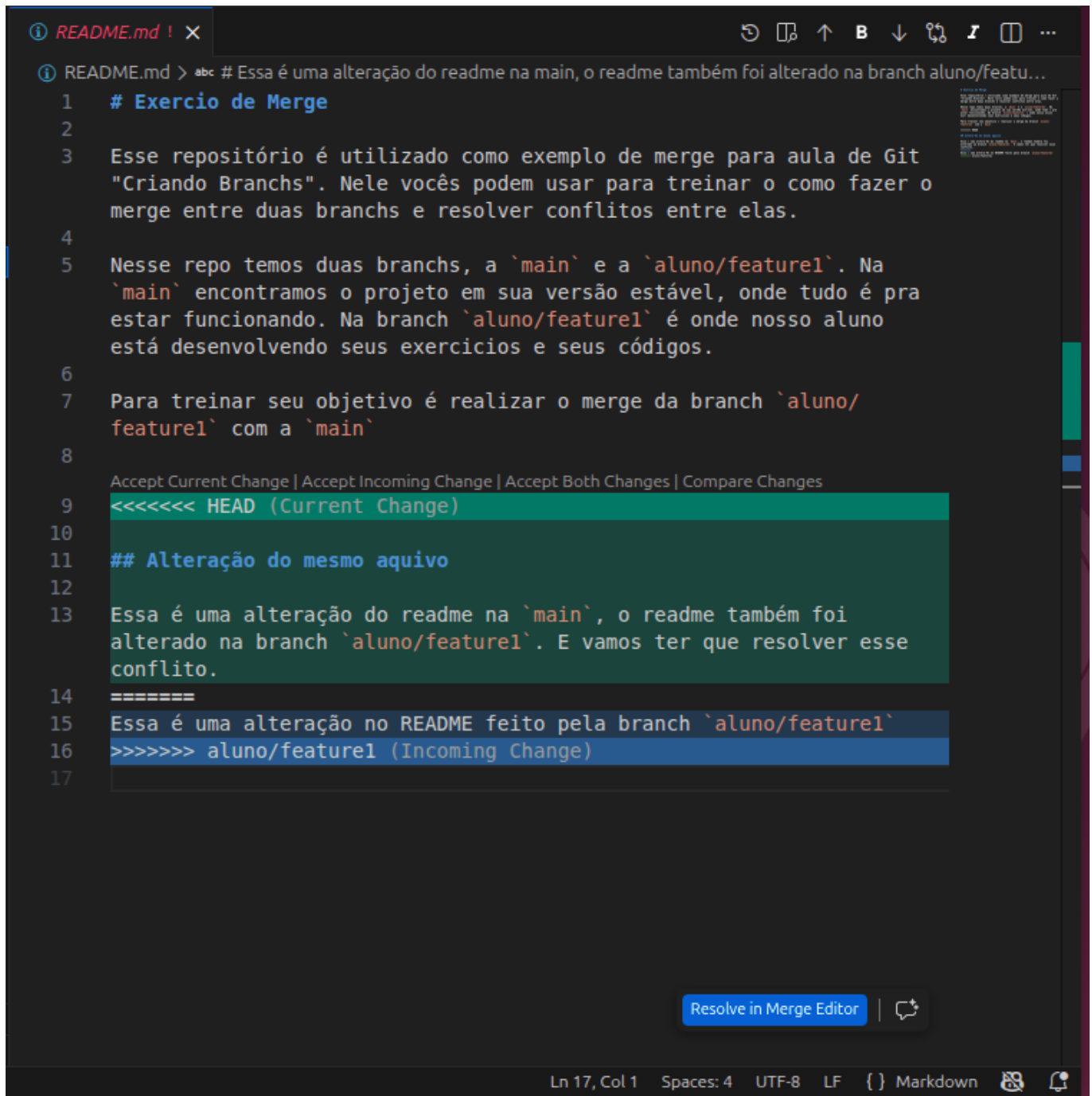
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line
```

Podemos ver que o Git indicou no arquivo onde está o conflito, através do <<<<<<< HEAD, ===== e do >>>>>>> aluno/feature1.

Entre o HEAD e os ===== ele indica qual é as alterações realizadas no HEAD que no caso é a nossa main. E entre o ===== e o aluno/feature1 está as alterações aplicadas na branch aluno/feature1.

Agora você deve editar o arquivo removendo as indicações do conflito e deixando apenas o código final atualizado sem conflito. Depois disso você deve usar o `git add` e depois realizar o commit.

DICA: Se você estiver usando extensões de Git no VScode fica muito mais facil resolver conflitos. Veja a imagem abaixo:



```
1 # Exercio de Merge
2
3 Esse repositório é utilizado como exemplo de merge para aula de Git
  "Criando Branchs". Nele vocês podem usar para treinar o como fazer o
  merge entre duas branchs e resolver conflitos entre elas.
4
5 Nesse repo temos duas branchs, a `main` e a `aluno/feature1`. Na
  `main` encontramos o projeto em sua versão estável, onde tudo é pra
  estar funcionando. Na branch `aluno/feature1` é onde nosso aluno
  está desenvolvendo seus exercicios e seus códigos.
6
7 Para treinar seu objetivo é realizar o merge da branch `aluno/
  feature1` com a `main`
8
  Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
9 <<<<<<< HEAD (Current Change)
10
11 ## Alteração do mesmo aquivo
12
13 Essa é uma alteração do readme na `main`, o readme também foi
  alterado na branch `aluno/feature1`. E vamos ter que resolver esse
  conflito.
14 =====
15 Essa é uma alteração no README feito pela branch `aluno/feature1`
16 >>>>>>> aluno/feature1 (Incoming Change)
17
```

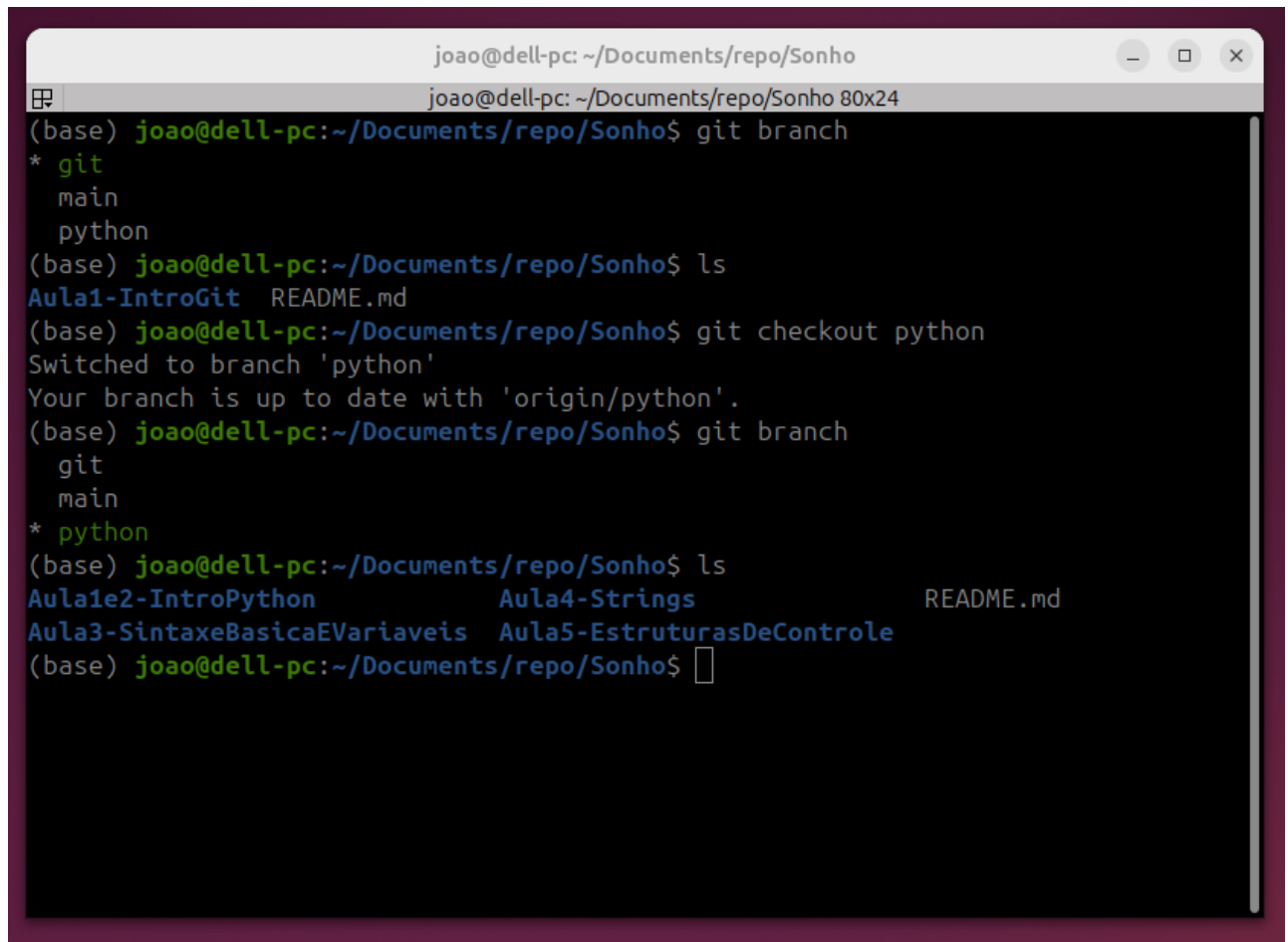
Além de uma indicação mais visual, a extensão já apresenta shortcuts para lidar com o conflito. Em cima da seção verde vocês podem observar as opções.

Exemplos

Nessa seção vou trazer situações que vocês podem encarar no dia a dia e que utilizam esses comandos Gits.

- **Navegando entre branches**

Como vocês já devem saber, as aulas do curso são organizadas no Github, e cada assunto tem a sua branch específico. Eu (Zezé) fiquei responsável tanto por Python quanto por Git, então pra mim o mais comum era ter que mudar de uma branch para outra:

A terminal window titled 'joao@dell-pc: ~/Documents/repo/Sonho' with a standard Linux prompt. The user runs 'git branch', showing 'git' as the current branch. Then 'ls' shows 'Aula1-IntroGit' and 'README.md'. Next, 'git checkout python' is run, switching to the 'python' branch. A final 'ls' shows a directory with files 'Aula1e2-IntroPython', 'Aula4-Strings', 'Aula3-SintaxeBasicaEVariaveis', 'Aula5-EstruturasDeControle', and 'README.md'.

```
joao@dell-pc: ~/Documents/repo/Sonho
joao@dell-pc: ~/Documents/repo/Sonho 80x24
(base) joao@dell-pc:~/Documents/repo/Sonho$ git branch
* git
  main
  python
(base) joao@dell-pc:~/Documents/repo/Sonho$ ls
Aula1-IntroGit  README.md
(base) joao@dell-pc:~/Documents/repo/Sonho$ git checkout python
Switched to branch 'python'
Your branch is up to date with 'origin/python'.
(base) joao@dell-pc:~/Documents/repo/Sonho$ git branch
  git
  main
* python
(base) joao@dell-pc:~/Documents/repo/Sonho$ ls
Aula1e2-IntroPython  Aula4-Strings  README.md
Aula3-SintaxeBasicaEVariaveis  Aula5-EstruturasDeControle
(base) joao@dell-pc:~/Documents/repo/Sonho$
```

- **Enviando branch nova para repositório remoto**

No curso trabalhamos com varios assuntos diferentes, e cada um tem a sua branch específica, supondo que desejo criar uma nova branch para outro assunto.

No primeiro comando eu crio a nova branch chamada teste, e no segundo eu envio ela para o repositório remoto. Assim se um novo assunto for criado no curso

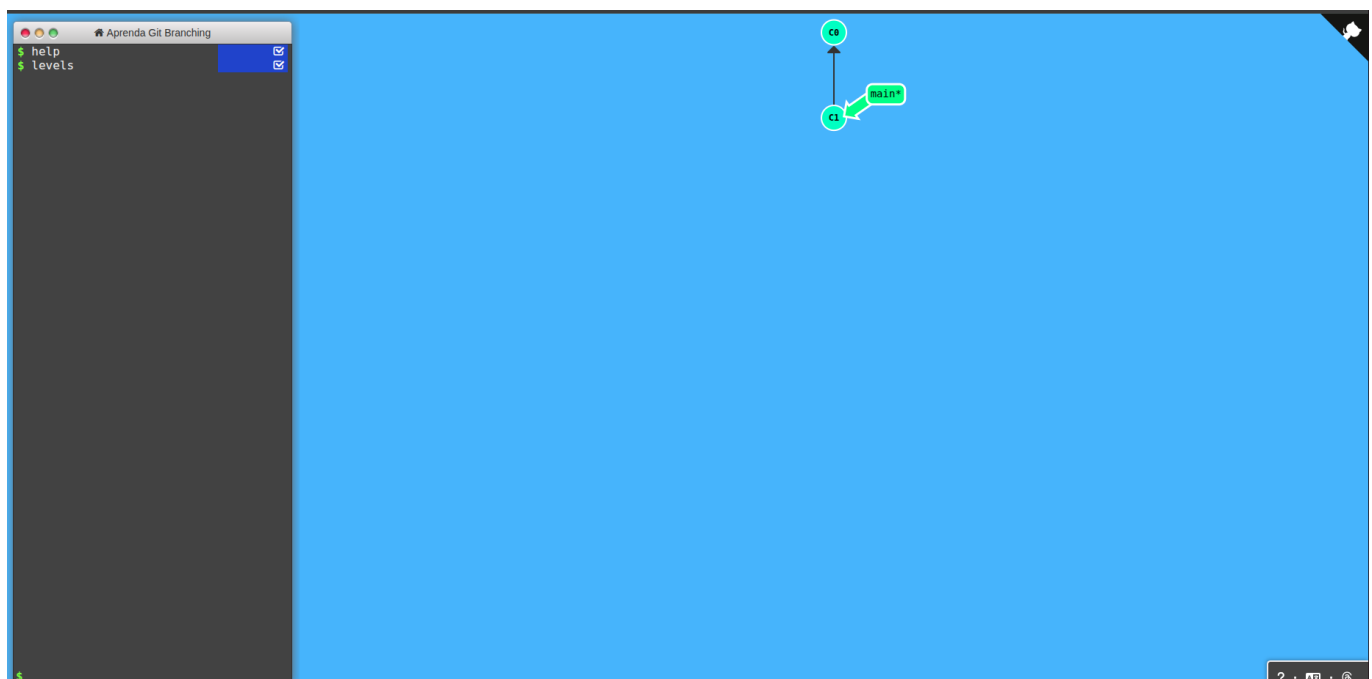
```
joao@dell-pc: ~/Documents/repo/Sonho
joao@dell-pc: ~/Documents/repo/Sonho 80x24
(base) joao@dell-pc:~/Documents/repo/Sonho$ git checkout -b teste
Switched to a new branch 'teste'
(base) joao@dell-pc:~/Documents/repo/Sonho$ git push -u origin teste
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'teste' on GitHub by visiting:
remote:   https://github.com/EESC-LabRoM/Sonho/pull/new/teste
remote:
To github.com:EESC-LabRoM/Sonho.git
 * [new branch]      teste -> teste
branch 'teste' set up to track 'origin/teste'.
(base) joao@dell-pc:~/Documents/repo/Sonho$
```

Atividade 01: Sequência Introdutória

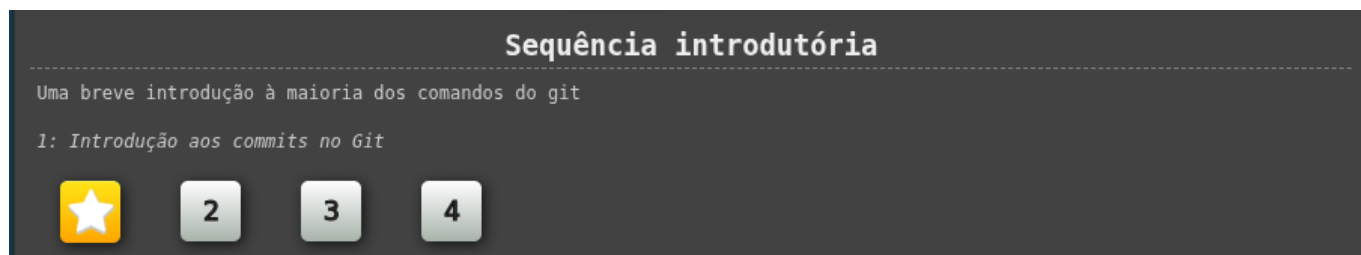
Para treinar o que aprendemos hoje vamos utilizar o site Learning Branching:

https://learngitbranching.js.org/?locale=pt_BR

Esse site não trabalha com os arquivos e códigos, mas é útil para entender e visualizar o que cada comando do Git realmente faz no fluxo de trabalho.



No site do [Learning Branches](#). Faça os três primeiros exercícios do primeiro modulo



Atividade 02: Realizando um merge de branches

Como falamos no começo da aula a ideia de branch é poder trabalhar paralelizado e também poder trabalhar de forma paralela ao código central na **main**, onde mantemos o código estável do nosso projeto.

Mas depois que terminamos a implementação de uma feature, devemos realizar o merge das duas. Nesse seção preparamos um repositório no git para que vocês possam treinar o merge, o repositório está disponível em: https://github.com/LabRoM-Graduacao/exercise_gitmerge

```
joao@dell-pc: ~/Documents/repo/Sonho
joao@dell-pc: ~/Documents/repo/Sonho 80x24
(base) joao@dell-pc:~/Documents/repo/Sonho$ git checkout -b teste
Switched to a new branch 'teste'
(base) joao@dell-pc:~/Documents/repo/Sonho$ git push -u origin teste
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'teste' on GitHub by visiting:
remote:   https://github.com/EESC-LabRoM/Sonho/pull/new/teste
remote:
To github.com:EESC-LabRoM/Sonho.git
 * [new branch]      teste -> teste
branch 'teste' set up to track 'origin/teste'.
(base) joao@dell-pc:~/Documents/repo/Sonho$
```

Mão na massa

O Git é uma ferramenta que se aprende praticando, e eesta semana que vocês tiveram a primeira aula de Python e já tiveram aula de c++, para ajudar a treinar o uso do Git, vamos propor uma atividade prática utilizando branches. A ideia é que vocês comecem a aplicar o conceito de ramificação desde o início do curso, simulando um fluxo de trabalho próximo ao que acontece em projetos reais.

No repositório de vocês, a branch main deve ser usada como a branch principal do projeto. Nela ficarão reunidos os códigos finais tanto em Python quanto em C++. Essa branch deve sempre representar a versão organizada e estável do repositório.

Para cada nova lista de exercícios, vocês deverão criar novas branches específicas para cada linguagem de programação. Por exemplo, uma branch para os exercícios em Python e outra para os exercícios em C++.

```
aluno.sobrenome/cpp/lista-x
```

ou

```
aluno.sobrenome/python/lista-x
```

Todo o desenvolvimento da lista daquela semana deve ser feito nessas branches, sem modificar diretamente a main.

Ao final de cada semana, depois de concluírem os exercícios, vocês deverão fazer o merge dessas branches de desenvolvimento com a branch main. Dessa forma, o repositório vai sendo atualizado de forma organizada, permitindo acompanhar a evolução do código ao longo do tempo e reforçando o uso correto das branches no Git.